

Sí. Con lo que has definido ya se puede diseñar bien la base de datos.

Y te diría una cosa importante:

No guardes **type** dentro de una sola tabla si ya sabes que vas a tener **posts** y **projects**

Tienes dos opciones:

Opción A

Dos tablas:

- **posts**
- **projects**

Opción B

Una sola tabla:

- **contents**
y un campo **content_type = post | project**
-

Mi recomendación para tu caso

Una sola tabla **contents.**

Porque realmente comparten casi todo:

- título
- descripción
- imagen destacada
- contenido html
- estado
- fechas
- etiquetas
- autor
- slug
- publicación

La diferencia entre post y proyecto es de **clasificación**, no de estructura.

Eso te simplifica:

- CRUD
 - filtros
 - etiquetas
 - listados
 - panel admin
-

- búsquedas

Modelo recomendado

Tabla `users`

Para autores y roles.

```
CREATE TABLE users (  
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(120) NOT NULL,  
  email VARCHAR(160) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  role ENUM('admin', 'editor') NOT NULL DEFAULT 'editor',  
  is_active TINYINT(1) NOT NULL DEFAULT 1,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP  
);
```

Tabla `contents`

Aquí van posts y proyectos.

```
CREATE TABLE contents (  
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  content_type ENUM('post', 'project') NOT NULL,  
  title VARCHAR(180) NOT NULL,  
  slug VARCHAR(220) NOT NULL UNIQUE,  
  short_description VARCHAR(320) NOT NULL,  
  cover_image_url VARCHAR(500) NOT NULL,  
  content_html LONGTEXT NULL,  
  status ENUM('draft', 'published', 'scheduled') NOT NULL DEFAULT 'draft',  
  published_at DATETIME NULL,  
  scheduled_at DATETIME NULL,  
  created_by INT UNSIGNED NOT NULL,  
  updated_by INT UNSIGNED NULL,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  CONSTRAINT fk_contents_created_by FOREIGN KEY (created_by) REFERENCES  
  users(id),  
  CONSTRAINT fk_contents_updated_by FOREIGN KEY (updated_by) REFERENCES  
  users(id),  
  INDEX idx_contents_type (content_type),  
  INDEX idx_contents_status (status),
```

```
INDEX idx_contents_created_at (created_at),
INDEX idx_contents_published_at (published_at),
INDEX idx_contents_title (title)
);
```

Tabla `tags`

Etiquetas reutilizables.

```
CREATE TABLE tags (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(80) NOT NULL UNIQUE,
  slug VARCHAR(100) NOT NULL UNIQUE,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Tabla pivote `content_tags`

Relación N:M entre contenidos y etiquetas.

```
CREATE TABLE content_tags (
  content_id INT UNSIGNED NOT NULL,
  tag_id INT UNSIGNED NOT NULL,
  PRIMARY KEY (content_id, tag_id),
  CONSTRAINT fk_content_tags_content FOREIGN KEY (content_id) REFERENCES
contents(id) ON DELETE CASCADE,
  CONSTRAINT fk_content_tags_tag FOREIGN KEY (tag_id) REFERENCES tags(id) ON
DELETE CASCADE
);
```

Tabla `content_files`

Para archivos adjuntos si luego quieres enlazar PDFs, docs, zip, etc.

```
CREATE TABLE content_files (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  content_id INT UNSIGNED NOT NULL,
  file_name VARCHAR(255) NOT NULL,
  file_path VARCHAR(500) NOT NULL,
  file_type VARCHAR(120) NULL,
  file_size INT UNSIGNED NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
CONSTRAINT fk_content_files_content FOREIGN KEY (content_id) REFERENCES
contents(id) ON DELETE CASCADE,
INDEX idx_content_files_content_id (content_id)
);
```

Por qué esta estructura es la buena

Ventajas

1. Un solo CRUD real

No necesitas duplicar toda la lógica para **posts** y **projects**.

2. Filtros más simples

Tu listado puede pedir:

- por **content_type**
- por **tag**
- por fecha
- por nombre

muy fácil.

3. Etiquetas compartidas

Una etiqueta puede servir tanto para un post como para un proyecto.

4. Escala mejor

Si mañana añades:

- **tutorial**
- **case-study**
- **research**

solo añades nuevos tipos, no nuevas tablas.

Qué campos cubren tu necesidad

Para la card

- **title**
- **short_description**
- **cover_image_url**
- tags relacionadas

Para la vista detalle

- `title`
- `cover_image_url`
- `content_html`
- `published_at`
- `tags`
- `autor`

Para administración

- `content_type`
- `status`
- `created_at`
- `updated_at`
- `scheduled_at`

Reglas de negocio

Estado

- `draft`
- `published`
- `scheduled`

Fechas

- `created_at`: cuando se crea
- `updated_at`: cuando se edita
- `published_at`: cuando se publica realmente
- `scheduled_at`: cuando debe publicarse si está programado

Lógica recomendada

Si `status = draft`

- `published_at = NULL`
- `scheduled_at = NULL` o puede quedar null

Si `status = published`

- `published_at = NOW()`
- `scheduled_at = NULL`

Si `status = scheduled`

- `scheduled_at` obligatorio

- `published_at = NULL` hasta que se publique

API que te va a pedir esto

CRUD contenidos

```
GET    /api/contents
GET    /api/contents/:id
GET    /api/contents/slug/:slug
POST   /api/contents
PUT    /api/contents/:id
DELETE /api/contents/:id
```

Filtros

```
GET /api/contents?type=post
GET /api/contents?type=project
GET /api/contents?status=published
GET /api/contents?tag=angular
GET /api/contents?search=dashboard
GET /api/contents?from=2026-01-01&to=2026-12-31
```

Etiquetas

```
GET    /api/tags
POST   /api/tags
PUT    /api/tags/:id
DELETE /api/tags/:id
```

Usuarios

```
GET    /api/users
POST   /api/users
PUT    /api/users/:id
DELETE /api/users/:id
```

Te recomiendo añadir también `slug`

Muy importante para URLs limpias:

```
/posts/mi-primer-post  
/projects/sistema-de-dashboard-angular
```

Por eso lo he metido en [contents](#).

Ejemplo de registro real

```
INSERT INTO contents (  
  content_type,  
  title,  
  slug,  
  short_description,  
  cover_image_url,  
  content_html,  
  status,  
  published_at,  
  scheduled_at,  
  created_by  
) VALUES (  
  'post',  
  'Mi primer contenido',  
  'mi-primer-contenido',  
  'Esto es una breve descripción',  
  '/uploads/covers/mi-primer-contenido.png',  
  '<h2>Contenido generado con Quill</h2><p>Texto...</p>',  
  'draft',  
  NULL,  
  NULL,  
  1  
)
```

Resumen final de tablas

Mínimas

- [users](#)
- [contents](#)
- [tags](#)
- [content_tags](#)

Opcional desde ya

- `content_files`
-

Mi consejo

Empieza con estas 4:

- `users`
- `contents`
- `tags`
- `content_tags`

y deja `content_files` para el siguiente paso si quieres.

Orden correcto ahora

1. crear la base de datos
2. crear estas tablas
3. insertar usuario admin inicial
4. montar backend Express
5. crear endpoint `POST /api/contents`
6. crear endpoint `GET /api/contents`
7. luego subida de imagen

Si quieres, el siguiente mensaje te lo doy con un **archivo SQL completo listo para pegar en phpMyAdmin**, incluyendo:

- `CREATE DATABASE`
- `USE`
- todas las tablas
- índices
- un usuario admin de ejemplo
- etiquetas de ejemplo.